

BUSINESS REQUIREMENTS DOCUMENT (BRD)

and

DETAILED SOLUTION DESIGN (DSD)

End-to-End Cloud Data Engineering Pipeline

Local CSV → AWS S3 → Snowflake Bronze → Silver → Gold → PostgreSQL

Project Type: Data Engineering / Cloud ETL-ELT / Data Warehouse

Document Version: 1.0

Prepared For: Data Engineering Portfolio / Implementation

Primary Technologies: Python, AWS S3, Snowflake, SQL, PostgreSQL, Git/GitHub

Document Contents

1. 1. Executive Summary
2. 2. Business Requirements Document (BRD)
3. 3. Business Objectives
4. 4. Scope and Out of Scope
5. 5. Stakeholders and Roles
6. 6. Functional Requirements
7. 7. Non-Functional Requirements
8. 8. Business Rules
9. 9. Data Sources and Data Domains
10. 10. Data Quality Requirements
11. 11. Security and Compliance Requirements
12. 12. Operational and Monitoring Requirements
13. 13. Assumptions, Dependencies and Risks
14. 14. Acceptance Criteria
15. 15. Detailed Solution Design (DSD)
16. 16. End-to-End Architecture
17. 17. Component Design
18. 18. AWS S3 Design
19. 19. Snowflake Database and Layer Design
20. 20. Bronze Layer Design
21. 21. Silver Layer Design
22. 22. Gold Layer Design
23. 23. PostgreSQL Serving Layer
24. 24. Python Application Design
25. 25. ETL/ELT Processing Design
26. 26. Incremental Load and File Processing
27. 27. Error and Corrupt Data Handling
28. 28. Audit and Logging Design
29. 29. Security Design
30. 30. Deployment and CI/CD
31. 31. Testing Strategy
32. 32. Reconciliation and Control Framework
33. 33. Performance and Scalability
34. 34. Disaster Recovery and Recovery Procedures
35. 35. Implementation Plan
36. 36. Production Readiness Checklist
37. 37. Future Enhancements
38. 38. Glossary

1. Executive Summary

This project establishes an end-to-end batch data engineering platform for ingesting CSV files from a local machine, landing them in Amazon S3, loading raw data into Snowflake, transforming the data through Bronze, Silver and Gold layers, and publishing curated business-ready data to PostgreSQL. The solution is designed to demonstrate production-oriented data engineering practices including data validation, data quality, incremental processing, error handling, auditability, security, monitoring, reconciliation and controlled deployment.

2. Business Requirements Document (BRD)

2.1 Problem Statement

The organization receives business data as CSV files. A manual process for moving, validating, cleaning, transforming and distributing this data creates operational risk, inconsistent data, duplicate processing, poor traceability and delays in reporting. A controlled automated pipeline is required to create a reliable path from source files to analytical and operational consumers.

2.2 Proposed Business Solution

Implement a batch-oriented pipeline in which Python validates and uploads source files to AWS S3; Snowflake ingests raw data into the Bronze layer; SQL transformations clean and standardize data into Silver; business rules create Gold datasets; and Python/JDBC/connector-based processing loads selected Gold datasets into PostgreSQL for reporting, applications or downstream analytics.

3. Business Objectives

- Automate CSV ingestion from a controlled local source into cloud storage.
- Maintain an immutable/raw landing zone in AWS S3.
- Create a layered Snowflake architecture separating raw, cleansed and business-ready data.
- Improve data quality through validation, standardization, duplicate detection and business-rule checks.
- Prevent corrupted files and invalid records from contaminating curated datasets.
- Provide auditable file-level and record-level processing status.
- Support incremental processing so previously successful files are not unnecessarily reprocessed.
- Publish trusted Gold datasets into PostgreSQL for downstream consumption.
- Provide secure credential management and controlled access.
- Create a portfolio implementation that follows production-oriented engineering practices.

4. Scope and Out of Scope

In Scope	Out of Scope / Future
CSV ingestion and schema validation	Real-time streaming
Local-to-S3 upload	Enterprise MDM
S3 raw/archive/rejected zones	Advanced ML feature engineering
Snowflake Bronze/Silver/Gold	Full BI dashboard implementation
Data quality and reconciliation	Enterprise scheduler beyond initial orchestration
Gold-to-PostgreSQL publication	Multi-region active-active architecture
Logging, audit and error handling	Complex CDC from OLTP systems
Incremental file processing	Large-scale distributed compute beyond Snowflake

5. Stakeholders and Roles

Role	Responsibility
Business Owner	Defines business outcomes, rules and acceptance criteria.
Data Engineer	Builds ingestion, transformation, quality, orchestration and loading components.
Cloud Engineer / AWS Admin	Manages S3, IAM, networking and cloud security.
Snowflake Admin	Manages warehouse, database, roles, integrations and performance.
Database Admin	Manages PostgreSQL schemas, users, indexes and operational controls.

Role	Responsibility
QA / Data Quality	Validates functional, data and reconciliation requirements.
Consumer / Analyst	Consumes curated PostgreSQL/Gold datasets.

6. Functional Requirements

ID	Requirement	Description
FR-001	File discovery	System shall identify configured CSV files available for ingestion.
FR-002	File validation	System shall validate extension, file accessibility, header/schema and basic structural rules.
FR-003	S3 upload	Validated files shall be uploaded to the configured S3 raw path.
FR-004	File tracking	Each file shall receive a processing status such as RECEIVED, SUCCESS, FAILED or REJECTED.
FR-005	Bronze ingestion	Snowflake shall load raw source data with minimal business transformation.
FR-006	Silver transformation	Silver tables shall clean, standardize, deduplicate and validate data.
FR-007	Gold transformation	Gold datasets shall implement approved business calculations and aggregations.
FR-008	Data quality	Required data-quality checks shall run before data is promoted to downstream consumption.
FR-009	Corrupt handling	Invalid files/records shall be isolated and logged without silently entering curated layers.
FR-010	PostgreSQL publication	Approved Gold datasets shall be loaded into PostgreSQL using controlled load/upsert logic.
FR-011	Audit	The system shall capture row counts, timestamps, status and error details.
FR-012	Restartability	A failed run shall be restartable without creating uncontrolled duplicates.
FR-013	Archival	Successfully processed source files shall be archived according to retention policy.
FR-014	Monitoring	Pipeline execution and failure conditions shall be visible through logs/status tables.

7. Non-Functional Requirements

ID	Area	Requirement
NFR-001	Security	Credentials shall not be hard-coded or committed to source

ID	Area	Requirement
		control.
NFR-002	Reliability	Failures shall be captured with actionable error messages.
NFR-003	Idempotency	Re-running a successful file shall not create unintended duplicate business data.
NFR-004	Scalability	Design shall allow additional CSV domains and larger file volumes.
NFR-005	Maintainability	Configuration, SQL and Python code shall be separated.
NFR-006	Performance	Processing shall use set-based SQL and appropriate Snowflake warehouse sizing.
NFR-007	Auditability	Every run shall be traceable to source file(s) and target load.
NFR-008	Recoverability	Failed stages shall be recoverable from retained source/raw data.
NFR-009	Portability	Environment-specific values shall be externalized through configuration.
NFR-010	Observability	Logs and metrics shall support troubleshooting and operational reporting.

8. Business Rules

- Only approved CSV file patterns shall be eligible for processing.
- Mandatory business keys must not be NULL in Silver.
- Business keys must be unique where the source domain requires uniqueness.
- Text fields shall be trimmed and standardized.
- Email fields shall be normalized to lowercase and validated for basic format.
- Dates shall be converted to a standard date/timestamp representation.
- Negative or zero quantities shall be rejected where the business domain prohibits them.
- Only valid Completed orders shall contribute to sales metrics unless business rules explicitly state otherwise.
- Gold metrics shall be calculated only from quality-approved Silver records.
- A file marked SUCCESS shall not be loaded again unless an explicit reprocessing procedure is invoked.

9. Data Sources and Data Domains

Domain	Source File	Business Key	Purpose
Customer	customers.csv	customer_id	Customer master information
Product	products.csv	product_id	Product and pricing information
Order	orders.csv	order_id	Order header and status
Order Item	order_items.csv	order_item_id	Products and quantities within an order

10. Data Quality Requirements

DQ ID	Check	Example Rule	Action
DQ-001	Null key	customer_id must not be	Reject/quarantine

DQ ID	Check	Example Rule	Action
		NULL	
DQ-002	Duplicate key	One active customer per customer_id	Deduplicate/reject
DQ-003	Data type	ID and quantity must match expected type	Reject/quarantine
DQ-004	Email format	Email contains basic valid pattern	Flag/reject based on policy
DQ-005	Referential integrity	Order customer_id exists in customer master	Reject/flag
DQ-006	Positive quantity	quantity > 0	Reject record
DQ-007	Valid status	Status belongs to approved domain	Flag/reject
DQ-008	Date validity	Order date is parseable and within allowed range	Reject/flag
DQ-009	Row count reconciliation	Source rows equal expected loaded rows after rules	Alert
DQ-010	Freshness	File received within expected schedule	Alert

11. Security and Compliance Requirements

- Use AWS IAM users/roles with least-privilege permissions.
- Use Snowflake roles with least privilege for stages, schemas, tables and warehouses.
- Use PostgreSQL roles with only required SELECT/INSERT/UPDATE privileges.
- Store secrets in environment variables or a secrets manager; never hard-code passwords.
- Exclude .env, private keys and credentials from Git.
- Enable encryption at rest and TLS/in-transit encryption where supported.
- Avoid logging credentials or sensitive payloads.
- Apply retention and deletion policies appropriate to the data classification.

12. Operational and Monitoring Requirements

- Log pipeline start/end timestamps.
- Log file name, file size and checksum where practical.
- Log source, Bronze, Silver, Gold and PostgreSQL row counts.
- Log execution status for every pipeline stage.
- Capture error class, message and stage.
- Provide a control/audit table for historical run analysis.
- Alert on failure, zero-row unexpected loads, quality threshold breach and reconciliation mismatch.

13. Assumptions, Dependencies and Risks

Type	Item	Mitigation
Assumption	CSV layout is defined and version-controlled	Maintain schema/configuration files
Dependency	AWS account and S3 access	Create IAM policy and test connectivity
Dependency	Snowflake account	Configure database, warehouse, stage and integration
Dependency	PostgreSQL instance	Create database/schema/user and connectivity

Type	Item	Mitigation
Risk	Source schema changes	Schema validation and versioning
Risk	Duplicate files	File manifest/checksum control
Risk	Corrupted data	Reject/quarantine path and DQ checks
Risk	Credential exposure	Secrets manager/environment variables
Risk	Partial load	Transactions, staging tables and audit status
Risk	Large files	Partitioning, staged processing and warehouse sizing

14. Acceptance Criteria

39. A valid CSV can be uploaded from the local machine to the configured S3 raw location.
40. The same file can be ingested into Snowflake Bronze with an auditable load record.
41. Silver contains standardized and deduplicated data according to defined rules.
42. Invalid records/files are isolated and documented.
43. Gold contains approved business metrics and reconciles to Silver.
44. PostgreSQL contains the approved Gold dataset with correct row counts and totals.
45. A successful file is not unintentionally processed twice.
46. A failed pipeline creates logs/audit records sufficient for diagnosis and restart.
47. No credentials are exposed in source code or Git history.
48. The end-to-end pipeline can be demonstrated from source file through PostgreSQL.

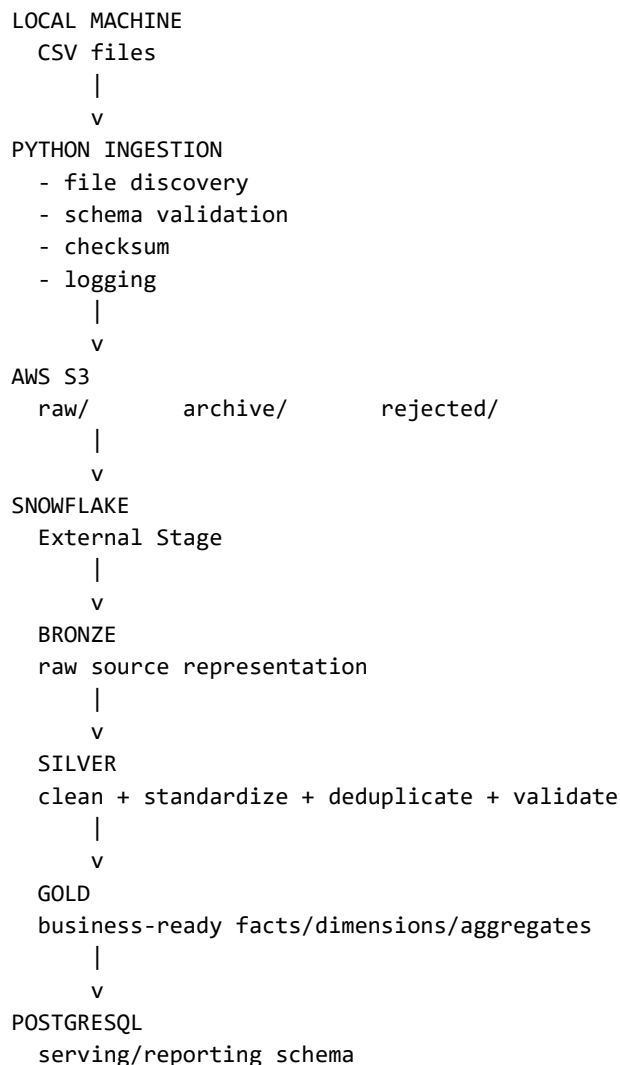
15. Detailed Solution Design (DSD)

15.1 Design Principles

- Separation of storage, raw ingestion, cleansing and serving responsibilities.
- Immutable raw data wherever possible.
- Idempotent processing.
- Configuration-driven file and table mappings.
- Set-based SQL transformations in Snowflake.
- Explicit data-quality gates between layers.
- Auditability at file, run and dataset level.
- Least privilege and secret isolation.
- Restartability and controlled reprocessing.

16. End-to-End Architecture

Logical flow: Local Machine → Python Validation → AWS S3 Raw → Snowflake External Stage → Bronze → Silver → Gold → PostgreSQL. A control/audit mechanism surrounds the pipeline to track file status, row counts, timings, quality outcomes and errors.



17. Component Design

Component	Technology	Responsibility
Ingestion Utility	Python + boto3	Discover, validate and upload files
Landing Storage	AWS S3	Durable raw, rejected and archive storage
Cloud Ingestion	Snowflake Stage/COPY INTO	Load raw files into Bronze
Transformation	Snowflake SQL	Silver cleansing and Gold business logic
Quality Framework	Snowflake SQL + Python	Validate data and thresholds
Serving Load	Python + psycopg2 / connector	Publish Gold data to PostgreSQL
Control Framework	Snowflake/PostgreSQL tables	Track runs, files and reconciliation
Version Control	Git/GitHub	Source code and SQL versioning
Scheduling	OS scheduler / enterprise scheduler	Automate execution

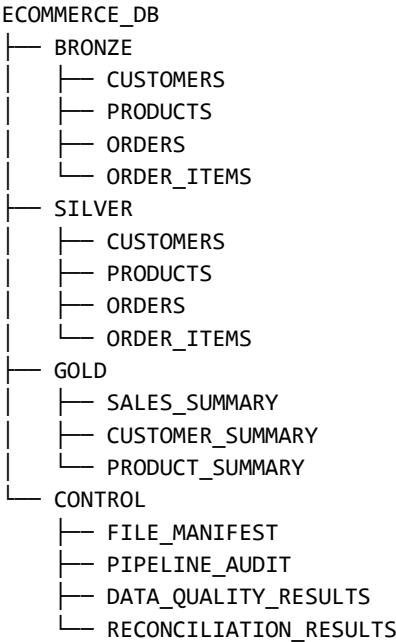
18. AWS S3 Design

Recommended bucket layout:

```
s3://<bucket>/
├── raw/
│   ├── customers/
│   ├── products/
│   ├── orders/
│   └── order_items/
├── rejected/
│   ├── customers/
│   ├── products/
│   ├── orders/
│   └── order_items/
└── archive/
    ├── customers/
    ├── products/
    ├── orders/
    └── order_items/
```

- Use date-based prefixes for higher volume, for example raw/orders/load_date=2026-09-14/.
- Enable versioning where appropriate.
- Apply lifecycle policies to archive/rejected data according to retention needs.
- Restrict write/read access through IAM policies.
- Prefer checksum or metadata-based duplicate detection.

19. Snowflake Database and Layer Design



20. Bronze Layer Design

Bronze is the raw ingestion layer. Its purpose is traceability rather than business correctness. Columns should closely match the source structure. Technical metadata is recommended.

Column	Type	Purpose
CUSTOMER_ID	NUMBER	Source customer identifier
FIRST_NAME	VARCHAR	Raw first name
LAST_NAME	VARCHAR	Raw last name
EMAIL	VARCHAR	Raw email
CITY	VARCHAR	Raw city
STATE	VARCHAR	Raw state
COUNTRY	VARCHAR	Raw country
SOURCE_FILE_NAME	VARCHAR	Source lineage
LOAD_TIMESTAMP	TIMESTAMP	Ingestion timestamp
BATCH_ID	VARCHAR	Pipeline run identifier

21. Silver Layer Design

Silver provides conformed, cleaned and quality-controlled records. Transformations include trimming, case normalization, data type conversion, null handling, duplicate resolution, standardization and referential checks.

- Use business keys to identify duplicates.
- Use deterministic survivorship rules for duplicates where necessary.
- Preserve useful technical lineage columns.
- Do not silently discard records; rejected counts must be measurable.
- Use staging/intermediate CTEs or transient tables when transformations are complex.

22. Gold Layer Design

Gold contains datasets optimized for business consumption. The recommended model includes a sales summary fact-like dataset plus customer/product summaries.

Dataset	Grain	Important Measures
GOLD.SALES_SUMMARY	One product per order date	total_orders, total_quantity, total_sales
GOLD.CUSTOMER_SUMMARY	One customer	orders, quantity, revenue
GOLD.PRODUCT_SUMMARY	One product	orders, quantity, revenue

23. PostgreSQL Serving Layer

PostgreSQL is treated as the downstream serving database. It should receive only approved Gold datasets. A staging table can be used before an atomic merge/upsert into the final table.

Table	Grain	Use
sales_summary	Order date + product	Reporting and analytics
customer_summary	Customer	Customer performance
product_summary	Product	Product performance

24. Python Application Design

Recommended project structure:

```
project/
├── data/
├── scripts/
│   ├── main.py
│   ├── upload_to_s3.py
│   ├── snowflake_load.py
│   ├── data_quality.py
│   └── postgres_load.py
├── sql/
│   ├── 01_database_setup.sql
│   ├── 02_bronze.sql
│   ├── 03_silver.sql
│   ├── 04_gold.sql
│   └── 05_postgres.sql
├── config/
│   └── config.yaml
├── logs/
├── tests/
├── .env
├── .gitignore
├── requirements.txt
└── README.md
```

24.1 Python Module Responsibilities

Module	Responsibilities
main.py	Orchestrates stages and handles overall status.
upload_to_s3.py	Validates files, calculates optional checksum and uploads to S3.

Module	Responsibilities
snowflake_load.py	Loads files into Bronze and executes transformation SQL.
data_quality.py	Runs DQ rules and writes results.
postgres_load.py	Extracts Gold data and loads/upserts into PostgreSQL.
config loader	Reads environment/configuration safely.

25. ETL/ELT Processing Design

49. Initialize a unique BATCH_ID and create an audit record with STARTED status.
50. Discover eligible local CSV files.
51. Validate file extension, required headers, encoding and basic structural rules.
52. Calculate checksum where duplicate detection is required.
53. Upload valid files to the S3 raw zone.
54. Register the file in FILE_MANIFEST.
55. Trigger Snowflake Bronze ingestion.
56. Capture Bronze source/target row counts and COPY errors.
57. Execute Silver transformations and data-quality checks.
58. Stop promotion if critical DQ gates fail.
59. Execute Gold transformations.
60. Reconcile Gold totals with Silver inputs.
61. Load approved Gold datasets into PostgreSQL staging.
62. Merge/upsert PostgreSQL staging into serving tables.
63. Record final counts and status as SUCCESS or FAILED.
64. Archive successfully processed files.

26. Incremental Load and File Processing

Recommended FILE_MANIFEST design:

Column	Type	Description
FILE_ID	VARCHAR	Unique file identifier
FILE_NAME	VARCHAR	Source file name
S3_PATH	VARCHAR	S3 object path
FILE_SIZE	NUMBER	File size in bytes
CHECKSUM	VARCHAR	Duplicate detection hash
RECEIVED_AT	TIMESTAMP	Receipt timestamp
PROCESS_START	TIMESTAMP	Processing start
PROCESS_END	TIMESTAMP	Processing end
STATUS	VARCHAR	RECEIVED/SUCCESS/FAILED/REJECTED
BATCH_ID	VARCHAR	Associated pipeline run
ERROR_MESSAGE	VARCHAR	Failure details

Idempotency rule: if the same checksum/file identity has already reached SUCCESS, the ingestion step should skip it unless an explicit REPROCESS request is made.

27. Error and Corrupt Data Handling

Failure Type	Example	Handling
File-level	Unreadable file / wrong extension	Move to rejected and log failure
Schema-level	Missing required column	Reject file; do not load to curated layer

Failure Type	Example	Handling
Record-level	Invalid numeric ID	Quarantine invalid record where supported
Business-level	Quantity <= 0	Reject/flag record
Referential	Unknown customer_id	Flag/reject according to policy
Infrastructure	S3/Snowflake unavailable	Retry with backoff, then fail run
PostgreSQL	Connection/load error	Keep Gold intact; retry downstream load

28. Audit and Logging Design

Table	Key Information
PIPELINE_AUDIT	batch_id, stage, start/end time, status, error
FILE_MANIFEST	file identity, checksum, S3 path, status
DATA_QUALITY_RESULTS	rule, source count, failed count, threshold, status
RECONCILIATION_RESULTS	source count, target count, source total, target total, variance

Recommended log levels: INFO for normal progress, WARNING for threshold/quality anomalies, ERROR for failed stages, DEBUG only during development.

29. Security Design

- AWS: IAM least privilege for S3 access; avoid long-lived access keys where role-based access is available.
- Snowflake: separate roles for ingestion, transformation and consumption.
- PostgreSQL: separate ETL and read-only consumer roles.
- Secrets: use environment variables for local development and a managed secrets solution for production.
- Network: restrict database access to trusted sources where infrastructure permits.
- Git: .gitignore must include .env, credential files and generated logs containing sensitive data.
- Data: encrypt in transit and at rest; restrict sensitive columns if introduced later.

30. Deployment and CI/CD

65. Developer commits code and SQL to a feature branch.
66. Automated checks validate Python syntax, unit tests and SQL formatting where available.
67. Pull request is reviewed.
68. Deploy configuration separately for DEV, TEST and PROD.
69. Run database DDL/transformations using versioned migration scripts.
70. Run a controlled test file through the pipeline.
71. Promote to production after acceptance criteria pass.

Suggested environments: DEV → TEST/UAT → PROD.

31. Testing Strategy

Test Level	Examples
Unit	File parser, schema validator, checksum logic
Integration	Python → S3, S3 → Snowflake, Snowflake → PostgreSQL
Data Quality	Null, duplicate, type, referential and business-rule checks
Reconciliation	Counts and revenue totals across layers
Negative	Corrupt CSV, missing header, invalid credentials, unavailable endpoint

Test Level	Examples
Restartability	Fail after Bronze and restart without duplicate Gold/PostgreSQL records
Performance	Large CSV, multiple files, repeated batch loads
Security	Verify unauthorized roles cannot access restricted objects

32. Reconciliation and Control Framework

Reconciliation should compare expected and actual outcomes at each major boundary. At minimum, compare file row counts to Bronze accepted/rejected rows, Silver output to DQ-approved input, Gold totals to Silver source totals, and PostgreSQL totals to Gold.

Boundary	Control
Local → S3	File exists, size/checksum matches expected
S3 → Bronze	COPY status + loaded/rejected counts
Bronze → Silver	Input rows vs cleaned/rejected rows
Silver → Gold	Business total and row-count controls
Gold → PostgreSQL	Target counts and financial/quantity totals

33. Performance and Scalability

- Use Snowflake set-based SQL rather than row-by-row transformations.
- Select warehouse size based on workload; scale up only when measurement supports it.
- Process only new files using manifest/checksum controls.
- Use date-based S3 prefixes for larger volumes.
- Use PostgreSQL staging plus batch insert/upsert rather than one-row-at-a-time commits.
- Add indexes to PostgreSQL columns used for lookup, merge and reporting.
- Consider Snowflake clustering only when query volume and table size justify it.

34. Disaster Recovery and Recovery Procedures

- S3 raw files are the recovery source for replaying ingestion.
- Snowflake Bronze provides an additional raw/traceability layer.
- Gold transformations should be reproducible from retained Bronze/Silver data.
- PostgreSQL can be rebuilt from Gold if retention permits.
- Failed files remain visible in FILE_MANIFEST and rejected/archive locations.
- Reprocessing should use an explicit batch/file identifier and must preserve audit history.

35. Implementation Plan

Phase	Activities	Deliverable
Phase 1	Create CSV dataset, project structure and Git repository	Local source + repository
Phase 2	Configure AWS S3/IAM and Python upload	S3 raw landing
Phase 3	Create Snowflake DB/schemas/stage and Bronze tables	Bronze ingestion
Phase 4	Build Silver transformations and DQ	Clean/conformed layer
Phase 5	Build Gold business models	Curated layer
Phase 6	Configure PostgreSQL and serving tables	PostgreSQL target

Phase	Activities	Deliverable
Phase 7	Build orchestration, audit, retries and incremental control	End-to-end pipeline
Phase 8	Testing, documentation and deployment	Production-style project

36. Production Readiness Checklist

- ☐ AWS S3 bucket and IAM policy configured
- ☐ Snowflake database, schemas, warehouse and stage configured
- ☐ PostgreSQL database/schema/users configured
- ☐ Secrets removed from source code
- ☐ File schema validation implemented
- ☐ S3 raw/rejected/archive paths implemented
- ☐ Bronze ingestion tested
- ☐ Silver transformation tested
- ☐ Gold transformation tested
- ☐ Data-quality framework implemented
- ☐ Corrupt data handling tested
- ☐ File manifest and audit tables implemented
- ☐ Incremental/idempotent loading implemented
- ☐ PostgreSQL staging/upsert implemented
- ☐ Reconciliation checks implemented
- ☐ Logging and failure handling implemented
- ☐ Unit/integration/negative tests completed
- ☐ README and architecture documentation completed
- ☐ GitHub repository cleaned of secrets and unnecessary raw data

37. Future Enhancements

- Apache Airflow or another enterprise orchestrator for dependency management.
- Snowpipe/Snowpipe Streaming for event-driven or near-real-time ingestion.
- dbt for modular, tested and documented Snowflake transformations.
- AWS Lambda/EventBridge for event-driven file arrival processing.
- AWS Secrets Manager for production secrets.
- Terraform for infrastructure as code.
- Docker for reproducible local development.
- Power BI/Tableau for analytics on PostgreSQL or Gold.
- Data catalog and lineage tooling.
- CDC ingestion from operational databases.
- Automated data-quality alerting through email/Teams/Slack.

38. Glossary

Term	Meaning
ETL	Extract, Transform, Load
ELT	Extract, Load, Transform
S3	Amazon Simple Storage Service
Bronze	Raw/landing data layer
Silver	Cleaned and conformed data layer

Term	Meaning
Gold	Business-ready curated data layer
DQ	Data Quality
DWH	Data Warehouse
Idempotent	Repeated execution produces the same intended result
Batch ID	Unique identifier for one pipeline execution
Manifest	Control table tracking source files and processing status
Reconciliation	Comparison of expected and actual processing outcomes
Upsert	Insert new records and update existing records

Appendix A – Example Snowflake SQL Skeleton

```
CREATE DATABASE IF NOT EXISTS ECOMMERCE_DB;
```

```
CREATE SCHEMA IF NOT EXISTS ECOMMERCE_DB.BRONZE;
CREATE SCHEMA IF NOT EXISTS ECOMMERCE_DB.SILVER;
CREATE SCHEMA IF NOT EXISTS ECOMMERCE_DB.GOLD;
CREATE SCHEMA IF NOT EXISTS ECOMMERCE_DB.CONTROL;
```

```
-- Bronze: raw representation
```

```
CREATE TABLE IF NOT EXISTS ECOMMERCE_DB.BRONZE.CUSTOMERS (
    CUSTOMER_ID NUMBER,
    FIRST_NAME VARCHAR(100),
    LAST_NAME VARCHAR(100),
    EMAIL VARCHAR(200),
    CITY VARCHAR(100),
    STATE VARCHAR(100),
    COUNTRY VARCHAR(100),
    SOURCE_FILE_NAME VARCHAR(500),
    LOAD_TIMESTAMP TIMESTAMP_NTZ,
    BATCH_ID VARCHAR(100)
);
```

```
-- Silver example
```

```
CREATE OR REPLACE TABLE ECOMMERCE_DB.SILVER.CUSTOMERS AS
SELECT DISTINCT
    CUSTOMER_ID,
    TRIM(FIRST_NAME) AS FIRST_NAME,
    COALESCE(NULLIF(TRIM(LAST_NAME), ''), 'Unknown') AS LAST_NAME,
    LOWER(TRIM(EMAIL)) AS EMAIL,
    TRIM(CITY) AS CITY,
    TRIM(STATE) AS STATE,
    TRIM(COUNTRY) AS COUNTRY
FROM ECOMMERCE_DB.BRONZE.CUSTOMERS
WHERE CUSTOMER_ID IS NOT NULL;
```

```
-- Gold example
```

```
CREATE OR REPLACE TABLE ECOMMERCE_DB.GOLD.SALES_SUMMARY AS
SELECT
    O.ORDER_DATE,
    P.PRODUCT_ID,
    P.PRODUCT_NAME,
```

```

P.CATEGORY,
COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
SUM(OI.QUANTITY) AS TOTAL_QUANTITY,
SUM(OI.QUANTITY * P.PRICE) AS TOTAL_SALES
FROM ECOMMERCE_DB.SILVER.ORDERS O
JOIN ECOMMERCE_DB.SILVER.ORDER_ITEMS OI
  ON O.ORDER_ID = OI.ORDER_ID
JOIN ECOMMERCE_DB.SILVER.PRODUCTS P
  ON OI.PRODUCT_ID = P.PRODUCT_ID
WHERE O.STATUS = 'Completed'
GROUP BY
  O.ORDER_DATE,
  P.PRODUCT_ID,
  P.PRODUCT_NAME,
  P.CATEGORY;

```

Appendix B – Example End-to-End Run

1. Developer places orders_2026-09-14.csv in the configured local input folder.
2. Python validates the file and calculates a checksum.
3. Python uploads it to s3://<bucket>/raw/orders/.
4. FILE_MANIFEST records the file as RECEIVED.
5. Snowflake COPY INTO loads Bronze.
6. Bronze audit records source file and BATCH_ID.
7. Silver SQL standardizes and validates the records.
8. DQ checks calculate passed/failed counts.
9. Gold SQL calculates completed-order sales metrics.
10. Gold results are reconciled against Silver.
11. PostgreSQL staging receives Gold rows.
12. PostgreSQL serving table is merged/upserted.
13. Final audit status becomes SUCCESS.
14. Source file is moved/copied to archive.

Appendix C – Recommended Repository Deliverables

- README.md with architecture, setup and execution instructions
- BRD_DSD_Project_Document.docx
- Architecture diagram
- Sample CSV files
- Python ingestion and orchestration scripts
- Snowflake DDL and transformation SQL
- PostgreSQL DDL and loading SQL
- Data-quality SQL
- Configuration template without secrets
- .gitignore
- requirements.txt
- Test cases and sample invalid files
- Sample pipeline log and audit screenshots (with secrets removed)